# Asymptotic Complexity & Recurrence Relations

COMPLETE SOLUTIONS GUIDE • GATE & PSU LEVEL

## Q1 (GATE 2021 Style)

**Given Recurrence:**

$$T(n) = T(n/2) + T(2n/5) + 7n$$
$$T(0) = 1$$

**Solution:**

To find the asymptotic complexity, we can apply the **Akra-Bazzi method** for linear recurrences. The recurrence is of the form $T(n) = \Sigma\, a_i\, T(b_i\, n) + f(n)$, where $a_1 = 1$, $b_1 = 1/2$ and $a_2 = 1$, $b_2 = 2/5$.

First, we solve for $p$ in the characteristic equation:

$$(1/2)^p + (2/5)^p = 1$$

Let's test $p = 1$:

$$1/2 + 2/5 = 5/10 + 4/10 = 9/10 < 1$$

Since the sum is less than 1 when $p = 1$, the true value of $p$ must be strictly less than 1 ($p < 1$). The driving function is $f(n) = 7n = \Theta(n^1)$.

Since the exponent of the driving function (1) is strictly greater than $p$, the linear $7n$ term completely dominates the recurrence growth.

**Answer:** $\Theta(n)$

## Q2 (GATE 2021 Style)

**Given Expression:** For constants $a \geq 1$ and $b > 1$:

$$T(n) = aT(n/b) + f(n)$$

Under what conditions will $T(n) = \Theta(n \log n)$ hold?

**Solution:**

By Master's Theorem, for $T(n) = \Theta(n \log n)$ to hold, we look at Case 2 of the standard formulation. We require the critical exponent $\log_b a$ to equal 1, which means:

$$\log_b a = 1 \Rightarrow a = b$$

When $a = b$, the core work term is $n^{\log_b a} = n^1 = n$. For the overall complexity to acquire the logarithmic multiplier and become $\Theta(n \log n)$, the driving function $f(n)$ must match this bound.
**Required Conditions:** `a = b` and `f(n) = Θ(n)`

## Q3 (GATE 2024 Style)

**Problem:** An algorithm checks whether an array of size N is sorted by making a single pass and comparing only adjacent elements. What is the tightest asymptotic running time?

**Solution:**

- **Best Case:** If early termination is implemented, it can stop in $O(1)$ time if the first pair is out of order.
- **Worst Case / Verification:** To confidently verify that any general input array is fully sorted, the loop must execute all the way to the end, carrying out exactly $N - 1$ comparisons.

The tightest overall bound to accurately complete this task for an arbitrary array is determined by the linear worst-case pass.
**Answer:** `Θ(N)`

### Q4 (GATE 2015 Style)

**Problem:** Evaluate the asymptotic order of: $1^3 + 2^3 + 3^3 + \dots + n^3$. Which of the following are true?
[1] $\Theta(n^4)$ [2] $\Theta(n^5)$ [3] $O(n^5)$ [4] $\Omega(n^3)$

**Solution:**

The exact algebraic formula for the sum of cubes is:

$$\Sigma_{i=1}^{n} i^3 = [n(n+1)/2]^2 = (n^4 + 2n^3 + n^2) / 4 = \Theta(n^4)$$

Let's evaluate each option against this result:

- **$\Theta(n^4)$:** **True** (This is the exact, tight asymptotic bound).
- **$\Theta(n^5)$:** False (The polynomial order is lower than 5).
- **$O(n^5)$:** **True** (Since $n^4 \leq n^5$ asymptotically, it serves as a valid upper bound).
- **$\Omega(n^3)$:** **True** (Since $n^4 \geq n^3$ asymptotically, it serves as a valid lower bound).

**Correct Selections:** **$\Theta(n^4)$, $O(n^5)$, and $\Omega(n^3)$**

### Q5 (GATE 2015 Style)

**Given Code:**

```
int fun1(int n) {
    int i, j, k, p, q = 0;
    for(i = 1; i < n; ++i) {
        p = 0;
        for(j = n; j > 1; j = j / 2)
            ++p;
        for(k = 1; k < p; k = k * 2)
            ++q;
    }
    return q;
}
```

**Solution:**

1. **Outer Loop (i):** Runs from 1 to $n-1$. It iterates exactly $\Theta(n)$ times.
2. **First Inner Loop (j):** Instantiates at $n$ and continually divides by 2 until it reaches 1. The number of iterations is $\log_2 n$. Since $p$ increments each time, its final value is $p \approx \log_2 n$.
3. **Second Inner Loop (k):** Instantiates at 1 and repeatedly doubles until it hits $p$. This takes $\log_2 p$ steps. Substituting $p$ gives $\log_2(\log_2 n)$ iterations, incrementing $q$.

The final cumulative value of $q$ across all $n$ steps of the outer loop is:
**Answer:** $\Theta(n \log \log n)$

### Q6 (Classic GATE)

**Given Code:**

```
for(int i = n; i > 0; i /= 2) {
    for(int j = 0; j < i; j++) {
        // constant work
    }
}
```

**Solution:**

We trace the work done step-by-step as the outer loop index ***i*** decreases geometrically:

- When ***i*** = ***n***, the inner loop runs ***n*** times.
- When ***i*** = ***n/2***, the inner loop runs ***n/2*** times.
- When ***i*** = ***n/4***, the inner loop runs ***n/4*** times.

Summing this up gives a classic geometric progression series:

$$\textit{Total Work} = n + n/2 + n/4 + n/8 + \ldots + 1 = n\,(1 + 1/2 + 1/4 + \ldots) \approx 2n$$

**Answer:** $\Theta(n)$

### Q7 (Classic GATE)

**Given Code:**

```
for(int i = 1; i ≤ n; i++) {
    for(int j = 1; j ≤ i; j++) {
        // constant work
    }
}
```

**Solution:**

The inner loop executes exactly ***i*** times for each step of the outer loop:

$$\textit{Total Work} = 1 + 2 + 3 + \ldots + n = [n(n+1)] / 2$$

This arithmetic expansion yields an order of growth matching quadratic scale.

**Answer:** $\Theta(n^2)$

## Q8 (Recursion)

**Given Code:**

```
void fun(int n) {
    if(n ≤ 1) return;
    fun(n/2);
    fun(n/2);
}
```

**Solution:**

The recurrence relation representing the execution work is:

$$T(n) = 2T(n/2) + \Theta(1)$$

Applying Master's Theorem parameters: $a = 2, b = 2, f(n) = \Theta(1)$.
The critical comparison value is $n^{\log_b a} = n^{\log_2 2} = n^1$.
Since $f(n) = O(n^{1-\varepsilon})$ where $\varepsilon > 0$, this falls into Case 1, which means the leaf node work dominates.
**Answer:** $\Theta(n)$

## Q9 (Fibonacci Pattern)

**Given Code:**

```
int fib(int n) {
    if(n ≤ 1) return n;
    return fib(n-1) + fib(n-2);
}
```

**Solution:**

The recurrence relation is: $T(n) = T(n-1) + T(n-2) + \Theta(1)$.
Solving this linear homogeneous recurrence reveals a growth rate bound tightly to the golden ratio base $\Phi = (1 + \sqrt{5})/2 \approx 1.618$.
In standard multiple-choice examinations like GATE, this loose structural upper bound is often generalized exponentially.
**Answer:** $\Theta(\Phi^n)$ or loosely $\Theta(2^n)$

### Q10 (GATE Favorite)

**Problem:** Arrange the following functions in increasing order of growth rate:

$$n^{3/2},\quad n \log n,\quad n^{\log_2 n},\quad 2^n$$

**Solution:**

Let's evaluate their relative growth speeds asymptotically:

1. $n \log n$: Linearithmic growth (slowest here).
2. $n^{3/2} = n^{1.5}$: Polynomial growth (strictly faster than linearithmic).
3. $n^{\log_2 n}$: Quasi-polynomial growth, equivalent to $2^{(\log_2 n)^2}$.
4. $2^n$: Fully exponential growth (fastest here).

**Increasing Order:** $n \log n < n^{3/2} < n^{\log_2 n} < 2^n$

## Additional PSU-Level Questions

### Q11

**Recurrence:** $T(n) = T(n-1) + 1$

**Solution:**

Unrolling via backward substitution:
$T(n) = T(n-2) + 1 + 1 = T(n-k) + k$
Setting $k = n$ yields $T(n) = T(0) + n$.
**Answer:** $\Theta(n)$

### Q12

**Recurrence:** $T(n) = T(n-1) + n$

**Solution:**

Unrolling via backward substitution:
$T(n) = T(n-2) + (n-1) + n = T(0) + 1 + 2 + \ldots + n$
This sum forms a standard arithmetic progression matching the quadratic order.
**Answer:** $\Theta(n^2)$

## Q13

**Recurrence:** $T(n) = 2T(n-1) + 1$

**Solution:**

Unrolling via backward substitution:
$T(n) = 2(2T(n-2) + 1) + 1 = 4T(n-2) + 2 + 1 = 2^k T(n-k) + \Sigma_{i=0}^{k-1} 2^i$
Setting $k = n$ transforms the sum into a geometric series leading directly to $2^n - 1$.
**Answer:** $\Theta(2^n)$

## Q14

**Recurrence:** $T(n) = T(n/2) + 1$

**Solution:**

Applying Master's Theorem parameters: $a = 1, b = 2, f(n) = 1 = \Theta(n^0)$.
The critical comparison term is $n^{\log_2 1} = n^0 = 1$.
Since $f(n) = \Theta(n^{\log_b a})$, it matches Case 2 exactly, adding a logarithmic factor.
**Answer:** $\Theta(\log n)$

## Q15

**Given Code:**

```
for(int i = 1; i ≤ n; i *= 2) {
    for(int j = 1; j ≤ n; j++) {
        // constant work
    }
}
```

**Solution:**

- **Outer Loop (i):** Geometrically doubles each iteration step ($1, 2, 4, 8, \ldots$) up to $n$. This runs exactly $\Theta(\log n)$ times.
- **Inner Loop (j):** Completely independent of the outer index variable $i$, running exactly $n$ times per outer iteration.

Total operational complexity is simply the product of both loops' boundaries.
**Answer:** $\Theta(n \log n)$